

# Ch 05: Page Faults

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science  
East Texas A&M University

Fall 2026



**EAST TEXAS A&M**

- 1 Page Faults and Lazy Allocation
- 2 More OS Uses of Page Tables and PT Exceptions



# Ch 05: Page Faults and Lazy Allocation

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science  
East Texas A&M University

Fall 2026



**EAST TEXAS A&M**

# Page Fault Exception

- RISC-V CPU raises a page fault exception when
  - ① virtual address (VA) is used that has no mapping in the page table
  - ② VA has a mapping whose PTE\_V flag is clear
  - ③ VA permission (PTE\_R, PTE\_W, PTE\_X, PTE\_U) forbid the operation
    - load page fault (caused by load instructions not marked PTE\_R e.g. a read from a page not marked readable)
    - store page fault (a store on a page not marked PTE\_W)
    - instruction page fault (caused by fetches of instructions to be executed)
- scause register indicates type of page fault
- stval register contains address that caused fault

The combination of page tables and page faults is a powerful tool for an OS

- Give kernel level of indirection between virtual and physical addresses
- Kernel can control structure and content of address spaces
- Allow kernel to intercept loads and stores



Kernel can use these capabilities to increase efficiency

- copy-on-write fork allows kernel to transparently share memory between parent and child
- memory-mapped files, where kernel uses paging to cause a file's contents to appear in an application's address space
- lazy memory allocation, only allocate and load pages a program actually reads and writes
  - xv6 implements lazy allocation, but not the other two



# Lazy Allocation

xv6's **lazy allocation** has 2 parts

- 1 When an application calls `sbrk` with the flag `SBRK_LAZY` the kernel simply notes the increase in size of the process address space, but it does not allocate physical memory and does not create PTEs yet for the new range of virtual addresses.
- 2 On a page fault on one of these new addresses, kernel allocates a page of physical memory and maps it into the page table.

Advantages:

- Convenient for applications, they can request worst case usage but not pay the cost if don't need all that memory
- Without lazy allocation, all work to grow a lot of memory happens up front, can be notably bad performance hit
  - On the other hand, page faults incur extra overhead that, if the memory is needed, will be much more work than just doing the work up front.



# Summary

- RISC-V raises a page fault exception when VA is not yet mapped in page table, or when PTE\_V flag is clear, or when permissions don't allow the operation asked for
- Page tables and page table exceptions are tools OS uses to implement many features
  - virtual memory
  - lazy memory allocation
  - copy on write process forks
- xv6 implements lazy memory allocation
  - when application requests dynamic memory (using `sbrk`) can request `SBRK_LAZY` and kernel simply notes increase in virtual address space size
  - on reference to unmapped VA, xv6 allocates a page of physical memory and maps it into the page table



# Ch 05: More OS Uses of Page Tables and PT Exceptions

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science  
East Texas A&M University

Fall 2026



**EAST TEXAS A&M**



# Demand Paging

- In the `exec()` system call, xv6 loads all of an application's text and data into memory when starting the application.
- **Demand paging**: text and data pages for an application are not actually allocated and loaded into memory until an actual reference to data on the page is made by the application.
  - On `exec`, just create the user page table with all PTE's marked as invalid
  - OS starts the program running, each time a program uses a page for the first time a page fault occurs
  - Can be implemented transparently to the application
- **Paging to disk**: store only a fraction of user pages in RAM, rest in **paging area**
  - If a reference to a paged out page is made, a page fault exception occurs and the OS pages in the page, potentially selecting an existing page in memory to be **evicted** and paged back out to the paging area



# Memory-Mapped Files, Dynamic Stacks

Other examples of features that can be implemented using page tables and paging exceptions:

- Automatically extending stacks.
  - For example xv6 has a guard page to detect stack overflow.
  - Instead of panicking could handle the exception and allocate an additional page if stack overflows
- **Memory-mapped files:** files that a program maps into its address space
  - `mmap` system call
  - Can thus access data using load and store instructions rather than file system calls



# Copy-On-Write (COW) Fork

- `fork()` system call causes creation of new process, but initial content of memory is same as parents (instructions and data)
  - xv6 creates new page table, and allocates and copies parent memory for child address space
- **Copy-on-write** (COW) fork causes parent and child to safely share physical memory by appropriate use of page-table permissions and page faults
  - All pages are mapped to read only (PTE\_W flag is cleared)
  - New page table created for child, but just a copy of parent's page table, translating to shared pages
    - Need a new data structure that keeps track of number of entities sharing a page of memory
    - Need to also still be aware of which pages should not be writable before being shared and PTE\_W cleared
  - When either tries to write a shared page, the RISC-V raises a page-fault exception
    - OS then allocates a new page of physical memory
    - copies the contents of shared page to new page
    - performs the write
    - also keeps track of number of processes sharing a page, need to decrease



# Summary

- Copy-on-write fork have child page table initially sharing all references to parent pages
  - All pages are marked read only, so that page fault generated when a write is attempted
  - Only when a page is written will a copy be made so there are separate pages for parent and child
  - Since a lot of pages are never written, and also since often an application exec a new set of code and data to run, this can increase performance greatly for forking and creating new processes
- Demand paging is implemented by having the exec call not actually load all data and text into memory
  - Instead a page table is created with all pages set to invalid
  - First time each page is referenced, a page table exception occurs and the OS loads the page lazily
  - Can also be used in conjunction with paging to disk
- Automatically extend system and user stacks by guard page trick, but allocating new page if stack overflows
- Memory-mapped files to allow process to map file to address space



# References



**EAST TEXAS A&M**